SLIIT
Discover Your Future

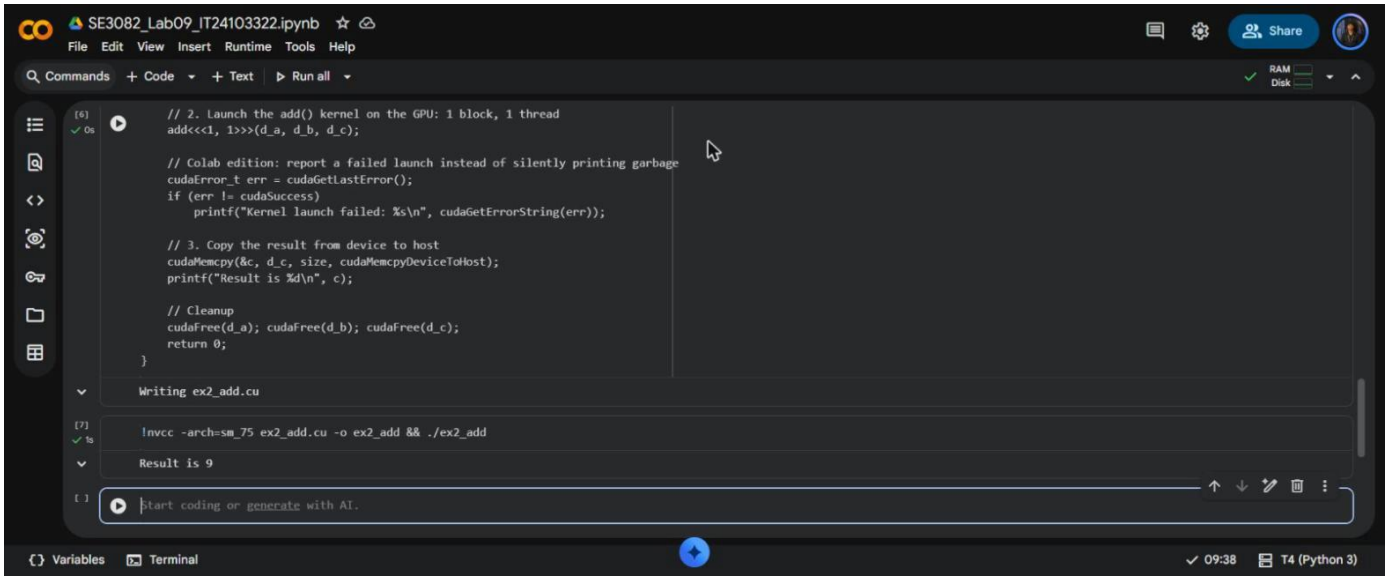
BSc (Hons) in Computer Science
Year 3

Lab Sheets

SE3082 – Parallel Computing **Semster 1, 2026**

IT24103322

Exercise 2



The program adds two integers on the GPU with one block of one thread. a = 2 and b = 7 are copied from the host to the device with cudaMemcpy(..., cudaMemcpyHostToDevice), the add kernel is launched with add<<<1, 1>>>, and the result is copied back with cudaMemcpy(..., cudaMemcpyDeviceToHost). Output: Result is 9. No “Kernel launch failed” line was printed, so cudaGetLastError() returned cudaSuccess.

1. **Why do we need separate variables d_a, d_b, d_c on the device? Why can the kernel not use a, b, c directly?**

   The CPU and the GPU have separate memories. a, b and c live in host (CPU) memory, and the kernel runs on the GPU, which can only read and write device memory. d_a, d_b and d_c point to memory allocated on the GPU with cudaMalloc, so the inputs have to be copied into them before the kernel runs and the result has to be copied back afterwards.

2. **In cudaMemcpy(d_a, &a, ...) we pass &a, but in the kernel launch we pass d_a. Why?**

   cudaMemcpy needs the address of the source and the destination. a is a normal int, so &a is its address in host memory, and d_a is already a pointer to device memory. The kernel takes pointers to device memory (int *a), so it is given d_a, the GPU address, and never a host address.

3. **How many threads run the add kernel? What would you change to run it with 10 threads, and would the answer change?**

   add<<<1, 1>>> runs exactly one thread. To run it with 10 threads the launch becomes add<<<1, 10>>>. The answer stays 9, because every thread does the same *c = *a + *b with the same inputs and writes the same value to the same address; the extra threads only repeat the same work.